

SKILLOPT: Executive Strategy for Self-Evolving Agent Skills

Yifan Yang^{1,*,\dagger} Ziyang Gong^{2,*} Weiquan Huang^{3,*} Qihao Yang^{2,*} Ziwei Zhou^{4,*}
Zisu Huang^{4,*} Yan Li² Xuemei Gao¹ Qi Dai¹ Bei Liu¹
Kai Qiu¹ Yuqing Yang¹ Dongdong Chen¹ Xue Yang^{2,\dagger} Chong Luo¹

¹ Microsoft ² Shanghai Jiao Tong University ³ Tongji University ⁴ Fudan University

Agent skills today are hand-crafted, generated one-shot, or **evolved through loosely controlled self-revision**—none of which behaves like a deep-learning optimizer for the skill, and none of which reliably improves over its starting point under feedback. We argue the skill should instead be *trained* as the external state of a frozen agent, with the same discipline that makes weight-space optimization reproducible. SKILLOPT is, to our knowledge, the first systematic *controllable* text-space optimizer for agent skills: a separate optimizer model turns scored rollouts into bounded add/delete/replace edits on a single skill document, and an edit is accepted only when it strictly improves a held-out validation score. A textual learning-rate budget, rejected-edit buffer, and epoch-wise slow/meta update make skill training stable while adding zero inference-time model calls at deployment. Across six benchmarks, seven target models, and three execution harnesses (direct chat, Codex, Claude Code), SKILLOPT is best or tied on **all 52 evaluated (model, benchmark, harness) cells** and beats every per-cell competitor among human, one-shot LLM, Trace2Skill, TextGrad, GEPA, and EvoSkill skills. On GPT-5.5 it lifts the average no-skill accuracy by +**23.5** points in direct chat, by +24.8 inside the Codex agentic loop, and by +19.1 inside Claude Code. Transfer experiments further show that optimized skill artifacts retain value when moved across model scales, between Codex and Claude Code execution environments, and to a nearby math benchmark without further optimization.

Code: <https://aka.ms/SkillOpt>

Correspondence: yifanyang@microsoft.com, yangxue2019-sjtu@sjtu.edu.cn

* Equal contribution. ^{\dagger} Corresponding authors.

1 Introduction

Frontier language models are increasingly deployed as agents, from single-prompt callers to multi-step execution harnesses with tools, files, and verifiers [1–4]. In such settings, domain adaptation is no longer only about model weights or prompts: it also requires improving the *procedures* by which the agent gathers evidence, calls tools, follows domain conventions, and formats outputs [5, 6]. Agent skills provide a natural interface for this procedural adaptation [7, 8]: a skill is a portable natural-language artifact that packages procedures, domain heuristics, tool policies, output constraints, and failure modes, letting a frozen agent adapt through external text.

If the recurring object of adaptation is the agent’s procedure, the skill document itself should be trainable. Yet weight adaptation is often unavailable for closed frontier models and expensive

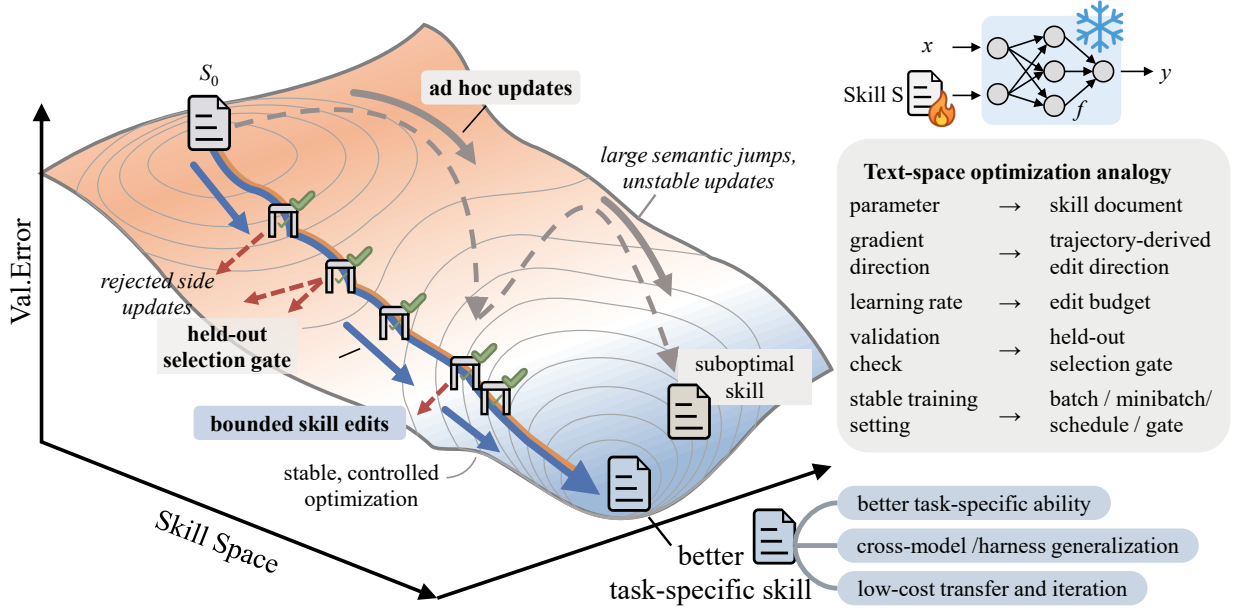


Figure 1 Overview of SKILLOPT. The target model executes tasks with a current skill, an additional frontier optimizer model converts trajectories into bounded add/delete/replace skill edits, and a held-out gate accepts only edits that improve validation performance. Accepted edits are exported as a reusable skill artifact, while rejected edits become negative feedback for later updates.

for open ones, while manually written or one-shot skills are brittle under a target domain or harness. Recent systems convert execution experience into reusable textual artifacts—distilling trajectory lessons, refining skill folders via failure analysis, building domain-specific skill libraries, or optimizing prompts from trajectory feedback [9–13]—but leave open a more basic question: if skills are the adaptation layer, how should they be optimized? Our key idea is to treat skill editing as a controllable domain-adaptation process, with the skill document as the external state, an additional frontier model as the optimizer, and training-style controls over evidence, step size, validation, and update direction.

We introduce SKILLOPT, a text-space optimizer for agent skills. Given a target domain, an initial skill, and the model being adapted, SKILLOPT repeatedly samples trajectory batches, analyzes successes and failures, and asks a frontier optimizer model to propose structured add/delete/replace edits. It then aggregates and ranks candidate edits under a textual learning-rate budget, applies a bounded update to the skill document, and evaluates the candidate skill on a held-out selection split before accepting it. Rejected edits are retained as negative feedback, while the epoch-wise slow/meta update preserves longer-horizon regularities. Figure 1 gives a schematic view of this loop. The deployed output is a compact `best_skill.md` file of roughly 300–2,000 tokens, with the adapted model and execution harness remaining fixed.

The deep-learning analogy is operational rather than decorative. Rollout and reflection batch sizes control the noise in the evidence used for each edit; the textual learning rate and schedule control how far one skill version is allowed to move from the previous one; the held-out gate plays the role of validation; and the epoch-wise slow/meta update acts like a momentum term, carrying stable editing directions across epochs. This stability is crucial: if consecutive skill revisions move too far or in inconsistent directions, rejected edits and previous accepted edits no longer provide a meaningful optimization history. With bounded, validation-gated updates, each revision remains close enough to the last one that later optimizer calls can learn from what helped, what failed, and what should be preserved.

We conduct, to our knowledge, the first systematic study of skill optimization as a domain-adaptation training method for frontier agents. We evaluate SKILLOPT on six benchmarks covering QA, spreadsheets, documents, math, and embodied decision making, across seven target models

from frontier-scale GPT to small-scale Qwen, and under three execution modes (direct chat, Codex harness, Claude Code harness). Out of 52 evaluated (model, benchmark, harness) cells, SKILLOPT is the best or tied-best measured method on all 52. With GPT-5.5 in direct chat, it lifts SearchQA from 77.7 to 87.3, SpreadsheetBench from 41.8 to 80.7, OfficeQA from 33.1 to 72.1, DocVQA from 78.8 to 91.2, LiveMathematicianBench from 37.6 to 66.9, and ALFWorld from 83.6 to 95.5 (a +23.5 point average gain over no skill), and it also beats the strongest *per-cell* baseline drawn from human-written, one-shot LLM, Trace2Skill, TextGrad, GEPA, and EvoSkill skills by +5.4 points on average. The same optimization interface is effective inside Codex-style and Claude Code-style execution loops, lifting GPT-5.5 by +24.8 and +19.1 points over no skill respectively, and outperforming EvoSkill by +14.0 and +3.2 points.

The learned artifacts also transfer beyond the exact training setting. A SpreadsheetBench skill trained on GPT-5.4 improves every smaller GPT variant we test; a Codex-trained spreadsheet skill transfers to Claude Code with a +59.7 point gain; and an OlympiadBench skill yields positive gains on Omni-MATH [14]. These transfer results are important for the paper’s application value: a skill can be optimized once, audited as text, and reused across related models, harnesses, or tasks without changing model weights. Our ablations explain why this works. Bounded textual learning outperforms uncontrolled rewriting, held-out gating prevents harmful proposals from accumulating, the rejected-step buffer converts failed edits into negative feedback, and the epoch-wise slow/meta update improves long-horizon refinement without bloating the deployed skill. Finally, per-benchmark case studies show that the learned skills remain compact (300–2,000 tokens after only 1–4 accepted edits), inspectable, and procedural rather than instance-specific.

Our contributions are as follows:

- We formulate agent-skill learning as optimization over an external natural-language state and introduce SKILLOPT, a harness-agnostic optimizer with rollout batches, reflection minibatches, add/delete/replace edits, textual learning rates, schedules, held-out acceptance, rejected-edit buffers, and epoch-wise slow/meta update.
- We provide a broad empirical study across six benchmarks, seven target models, and three execution harnesses, showing that SKILLOPT is best or tied-best on 52 of 52 cells and outperforms no-skill, human-skill, one-shot LLM-skill, prompt-optimization (TextGrad, GEPA), and skill-evolution (Trace2Skill, EvoSkill) baselines under every model.
- We validate the optimization design through component ablations and three forms of transfer (cross-model, cross-harness, cross-benchmark), showing that the exported skill artifact is compact, reusable, and deployable without model-weight updates.

2 Related Work

Prompt auto tuning and agent-configuration search. GEPA demonstrates that trajectory feedback can guide reflective prompt evolution and outperform reinforcement learning on several language-agent tasks [13]. ABSTRAL and EvoTest extend this idea from single prompts to multi-agent design documents and test-time agentic system evolution without gradients or fine-tuning [15, 16]. By treating language artifacts as optimizable objects, these methods can directly exploit execution feedback, but they mainly target prompts, system designs, or full configurations rather than reusable domain adaptation. SKILLOPT instead optimizes a persistent skill document that can be trained, validated, exported, and reused with the adapted model, applying language-level controllability to a stable procedural skill state.

Skill construction and skill evolution. SkillsBench and the SoK on agentic skills frame skills as reusable procedural knowledge, covering tool policies, applicability conditions, execution

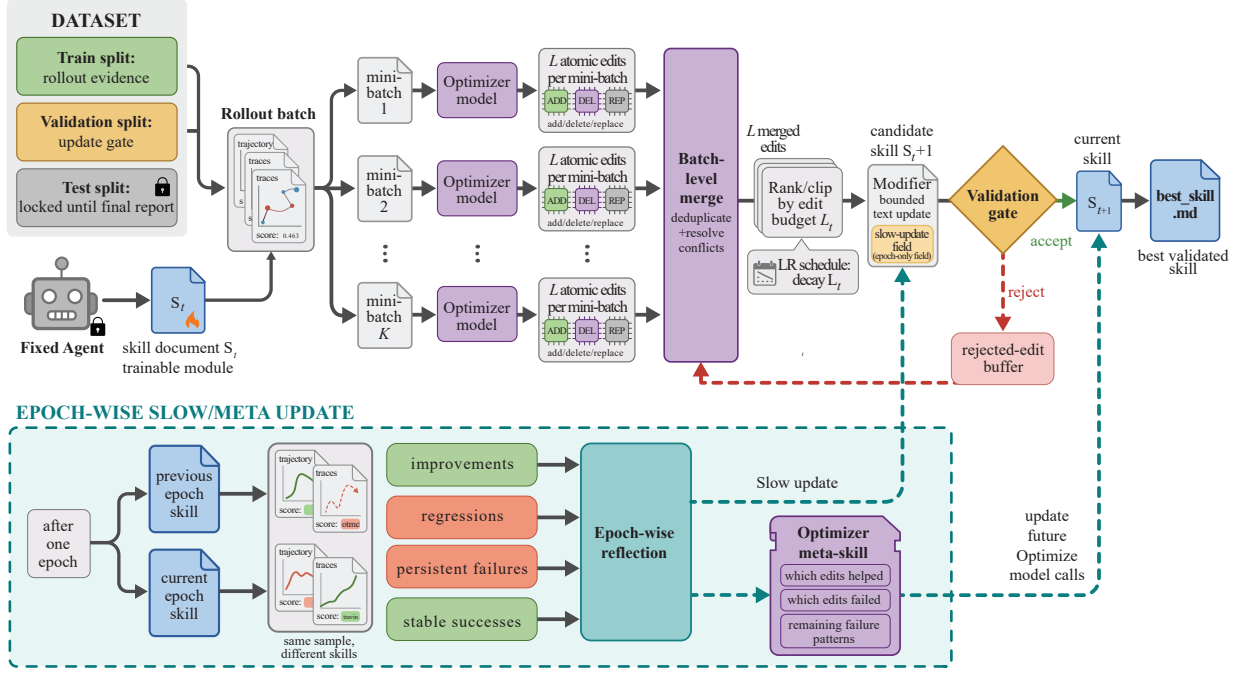


Figure 2 Pipeline of SKILLOPT. A frozen target model executes a rollout batch with the current skill; an optimizer model performs minibatch reflection over successes and failures, proposes bounded add/delete/replace edits, merges and ranks them under a scheduled edit budget, and accepts the candidate skill only through a held-out validation gate. Across epochs, the slow/meta update retains longer-horizon lessons without changing the target model.

routines, and supporting resources [7, 8]. Prior systems construct such skills from lifelong experience, trajectory lessons, skill knowledge bases, or heterogeneous domain resources [9, 12, 17–19], and further refine them through failure analysis, creation-evaluation-revision loops, co-evolving generators and verifiers, collective updates, or reinforcement learning [10, 11, 20–26]. While these works emphasize skill discovery, repository growth, sharing, evolutionary search, or policy optimization, SKILLOPT studies a narrower problem: how to train one compact domain skill with deep-learning-style controls such as trajectory batches, reflection minibatches, textual learning rates, validation gates, rejected-edit buffers, and slow/meta updates. This yields a controlled and auditable procedure for producing a portable `best_skill.md` without changing model weights.

3 Method

3.1 Problem Setup

A skill s is a natural-language policy inserted into the agent context before execution, consistent with recent work treating skills as reusable procedural knowledge for agents [7, 8]. In direct-chat benchmarks, it is prepended to the system or developer instruction; in tool-use harnesses, it becomes persistent procedural memory. We use M to denote the frozen target model whose behavior is being adapted through skill optimization. For a harness h , task x , and skill s , execution produces a trajectory τ and a scalar score r :

$$(\tau(s), r(s)) = h(M, x, s), \quad r(s) \in [0, 1]. \quad (1)$$

Given train, selection, and test splits $D_{\text{tr}}, D_{\text{sel}}, D_{\text{test}}$, SKILLOPT uses D_{tr} to generate a set of